

Simulación Termohidráulica de Tiempo Real en Rust

*Arquitectura de Componentes, Modelos Matemáticos y Código Fuente
para Sistemas 0D/1D Distribuidos*

Preparado para:
Biblioteca de Componentes Físicos
Orquestación por Diagrama de Bloques

Lenguaje de Implementación:
Rust (Edición 2021)
Rendimiento Monofásico Explícito

Índice

1. Componentes de Conexión y Distribución de Flujo (Resistencias)	3
1.1. Modelo Termohidráulico de Tubería 1D Discretizada	3
1.1.1. Topología del Modelo (Grilla Escalonada)	3
1.1.2. Estructura de Datos y Caché (SoA)	3
1.1.3. Estabilización Numérica	4
1.1.4. Implementación del Motor de Simulación (Método tick)	4
1.2. Bomba Centrífuga Dinámica	7
1.2.1. El Modelo Matemático de la Bomba Dinámica	7
1.2.2. Implementación de la Lookup Table 1D Lineal en Rust	7
1.2.3. Estructura e Implementación del Bloque Bomba	8
1.2.4. Acoplamiento de Energía (Transporte Térmico)	9
2. Componentes Volumétricos de Acumulación (Capacitancias)	10
2.1. Bloque Header (Volumen de Control Capacitivo)	10
2.1.1. Rol Arquitectónico y Topología	10
2.1.2. Modelado Matemático y Estrategia de Ablandamiento	10
2.1.3. Estructura de Datos en Rust	11
2.1.4. Acoplamiento de Frontera y Flujo Bidireccional	12
2.2. Open Tank (Tanque de Presión Hidrostática)	13
2.2.1. El Modelado Matemático	13
2.2.2. Estructura de Datos en Rust	13
2.2.3. Particularidades en la Red	14
2.3. Tank Closed (Tanque Presurizado de Nivel Variable)	16
2.3.1. El Modelado Matemático (0D de Dos Zonas)	16
2.3.2. Estructura de Datos en Rust	16
2.4. Tanque Térmico Estratificado	19
2.4.1. Topología de N Capas ($N = 2, 3, 4, \dots$)	19
2.4.2. Modelado de los Fenómenos de Mezclado	19
2.4.3. Implementación en Rust (Modelo de 3 Capas)	20

1. Componentes de Conexión y Distribución de Flujo (Resistencias)

1.1. Modelo Termohidráulico de Tubería 1D Discretizada

Este documento detalla la arquitectura, el modelado matemático y la implementación en Rust de un componente de tubería unidimensional (Pipe1D) basado en volúmenes finitos. El modelo está diseñado para simulaciones de tiempo discreto de fluidos monofásicos (como sistemas de refrigeración de reactores o circuitos de agua pesada), priorizando la estabilidad numérica, la velocidad de ejecución por ciclo y el manejo natural de flujos bidireccionales y de bajo caudal.

1.1.1. Topología del Modelo (Grilla Escalonada)

Para evitar la inestabilidad espacial que sufren los esquemas centrados, el Pipe1D utiliza una grilla escalonada (staggered grid). La tubería se divide en N volúmenes de control, lo que genera dos dominios de cálculo paralelos:

Celdas (N elementos): Representan los volúmenes finitos. Aquí residen las variables escalares: presión (p), entalpía (h) y densidad (ρ).

Caras o Uniones ($N + 1$ elementos): Representan las fronteras entre celdas (y los extremos conectados a los headers externos). Aquí reside la variable vectorial: el caudal másico (W).

Esta topología permite evaluar los gradientes de presión directamente sobre las caras, y los balances de masa directamente sobre las celdas.

1.1.2. Estructura de Datos y Caché (SoA)

Para maximizar el rendimiento del procesador y habilitar la vectorización, la memoria se organiza bajo el patrón SoA (Structure of Arrays). En lugar de tener un vector de objetos «Celda», se utilizan vectores pre-asignados y contiguos en memoria para cada propiedad física.

```
pub struct Pipe1D {
    pub n_cells: usize,

    // Geometría
    vol_cell: f64,
    inertia_gain: f64, // (dt * A) / dz
    fric_factor: f64, // (f * dz) / (2 * D * A^2)

    // Estado de las CARAS (Tamaño: N + 1)
    w: Vec<f64>, // Caudal actual (kg/s)
    w_last: Vec<f64>, // Memoria z^-1 del caudal (kg/s)

    // Estado de las CELDAS (Tamaño: N)
    p: Vec<f64>, // Presión (Pa)
    h: Vec<f64>, // Entalpía (J/kg)
    rho: Vec<f64>, // Densidad (kg/m³)
}
```

1.1.3. Estabilización Numérica

La simulación en tiempo discreto puro requiere estrategias específicas para evitar oscilaciones espurias (chattering) y explosiones por el límite CFL (Courant).

1.1.3.1. Momento: Fricción Semi-Implicita

A caudales muy bajos, el término de fricción cuadrática tradicional pierde su capacidad de amortiguación. Para garantizar la estabilidad incondicional de la ecuación de momento ante cualquier paso de tiempo Δt , se emplea una linealización semi-implícita de la fricción:

$$W^{n+1} = \frac{W^n + \frac{\Delta t \cdot A}{\Delta z} \Delta P}{1 + \frac{\Delta t \cdot A}{\Delta z} K_{\text{fric}} \cdot \max(|W^n|, \varepsilon_w)}$$

Donde ε_w es un piso de regularización que impone un comportamiento de flujo laminar cerca del cero.

1.1.3.2. Energía: Esquema Upwind Suavizado

Para el transporte de energía, el modelo debe capturar la dirección del flujo. Si se utiliza un esquema Upwind rígido, el cruce por cero del caudal induce oscilaciones térmicas severas. Se implementa una banda de transición lineal alrededor del flujo nulo para promediar las entalpías de las celdas adyacentes.

1.1.3.3. Masa: Ablandamiento de Presión (Capacitancia Numérica)

Dado que los líquidos son débilmente compresibles, cambios microscópicos de masa generan saltos masivos de presión. La EDO de presión se evalúa de forma explícita expandiendo la derivada total de la densidad, pero incrementando virtualmente la compresibilidad isotérmica ($\partial \rho / \partial p$) para «ablandar» el acoplamiento:

$$dp = \frac{(W_{\text{in}} - W_{\text{out}}) \cdot \Delta t}{V_{\text{cell}} \cdot \left(\partial \rho / \partial p \right)_{\text{virtual}}}$$

1.1.4. Implementación del Motor de Simulación (Método tick)

El lazo de resolución principal ejecuta el avance temporal en tres fases secuenciales. Este método debe ser llamado por el orquestador principal (el sistema de bloques) pasándole las condiciones de borde provenientes de los headers externos.

```
impl PipeID {
    /// Avanza el estado interno de la tubería un paso de tiempo dt.
    /// Retorna los caudales en los extremos (w_in, w_out) para acoplar con los
    headers.
    pub fn tick(&mut self, p_in: f64, h_in: f64, p_out: f64, h_out: f64, dt: f64) -
    > (f64, f64) {

        // =====
        // FASE 1: ECUACIÓN DE MOMENTO (Resolución en N+1 Caras)
        // =====

        // Cara 0 (Frontera de entrada)
        let rho_f0 = self.rho[0];
        self.w[0] = self.calc_flow_semi_implicit(self.w_last[0], p_in, self.p[0],
```

```

rho_f0);

    // Caras internas (1 a N-1)
    for i in 1..self.n_cells {
        let rho_f = (self.rho[i-1] + self.rho[i]) * 0.5; // Densidad promediada
en la cara
        self.w[i] = self.calc_flow_semi_implicit(self.w_last[i], self.p[i-1],
self.p[i], rho_f);
    }

    // Cara N (Frontera de salida)
    let rho_fN = self.rho[self.n_cells - 1];
    self.w[self.n_cells] =
self.calc_flow_semi_implicit(self.w_last[self.n_cells], self.p[self.n_cells-1],
p_out, rho_fN);

    // =====
    // FASE 2: ENERGÍA Y MASA (Resolución en N Celdas)
    // =====

    // 2.a Calcular flujos de energía (Phi) en las caras con Upwind suavizado
    let mut phi = vec![0.0; self.n_cells + 1];

    // Phi en la entrada
    let h_face_0 = self.calc_h_face_smooth(self.w[0], h_in, self.h[0]);
    phi[0] = self.w[0] * h_face_0;

    // Phi en caras internas
    for i in 1..self.n_cells {
        let h_face = self.calc_h_face_smooth(self.w[i], self.h[i-1],
self.h[i]);
        phi[i] = self.w[i] * h_face;
    }

    // Phi en la salida
    let h_face_N = self.calc_h_face_smooth(self.w[self.n_cells],
self.h[self.n_cells - 1], h_out);
    phi[self.n_cells] = self.w[self.n_cells] * h_face_N;

    // 2.b Resolver EDOs en las celdas
    for i in 0..self.n_cells {
        let mass = self.vol_cell * self.rho[i];

        // Balance universal: entra por la izquierda, sale por la derecha
        let net_energy_flow = phi[i] - phi[i+1];
        let net_mass_flow = self.w[i] - self.w[i+1];

        // Integración de Energía
        let dh = net_energy_flow * dt / mass;
        self.h[i] += dh;

        // Integración de Presión (Ablandada)
        // drho_dp virtual: ajustar en función de la estabilidad requerida
(CFL)

```

```

        let drho_dp_virtual = 4.5e-10 * self.rho[i] * (self.n_cells as f64);
        let dp = net_mass_flow * dt / (self.vol_cell * drho_dp_virtual);
        self.p[i] += dp;

        // Actualización Termodinámica (IAPWS-IF97 Región 1)
        self.rho[i] = self.get_density_if97(self.p[i], self.h[i]);
    }

    // =====
    // FASE 3: ACTUALIZACIÓN DE ESTADO z^-1
    // =====
    self.w_last.copy_from_slice(&self.w);

    (self.w[0], self.w[self.n_cells])
}

// Funciones auxiliares (Inlining forzado para máxima performance)

#[inline(always)]
fn calc_flow_semi_implicit(&self, w_prev: f64, p_up: f64, p_down: f64, rho:
f64) -> f64 {
    let delta_p = p_up - p_down;
    let numerator = w_prev + self.inertia_gain * delta_p;
    let w_mod = f64::max(w_prev.abs(), 1e-4);
    let denominator = 1.0 + self.inertia_gain * (self.fric_factor / rho) *
w_mod;
    numerator / denominator
}

#[inline(always)]
fn calc_h_face_smooth(&self, w: f64, h_up: f64, h_down: f64) -> f64 {
    let epsilon_w = 1e-4;
    if w > epsilon_w {
        h_up
    } else if w < -epsilon_w {
        h_down
    } else {
        let factor = (w + epsilon_w) / (2.0 * epsilon_w);
        factor * h_up + (1.0 - factor) * h_down
    }
}

#[inline(always)]
fn get_density_if97(&self, p: f64, h: f64) -> f64 {
    1000.0 // Valor simulado
}
}

```

1.2. Bomba Centrífuga Dinámica

Modelar una bomba centrífuga en tu esquema de tiempo discreto es sumamente elegante porque, desde el punto de vista topológico, la bomba se comporta exactamente igual que una cara (o junction) del caño: es una resistencia activa al flujo.

En lugar de que el motor del caudal sea únicamente la diferencia de presiones de los headers ($\Delta P = P_{\text{in}} - P_{\text{out}}$), la bomba añade un término de altura manométrica o salto de presión activo (ΔP_{bomba}) que depende del caudal actual y de la velocidad de giro de la bomba (N), determinado por tu lookup table.

Aquí tienes el diseño físico y la implementación en Rust para acoplarla de forma estable y bidireccional.

1.2.1. El Modelo Matemático de la Bomba Dinámica

La ecuación de momento discreta para el bloque bomba se deriva modificando la ecuación del caño. El salto de presión total que ve el fluido es el gradiente de los headers más la presión que inyecta el rodete, menos las pérdidas internas de la bomba:

$$W^{n+1} = W^n + \frac{\Delta t \cdot A}{L} (P_{\text{in}} - P_{\text{out}} + \Delta P_{\text{bomba}}(W^n, N) - \text{Fricción}(W^n))$$

1.2.1.1. Las Leyes de Afinidad para la Lookup Table

Por lo general, el fabricante entrega la curva de la bomba como una tabla de datos de Altura (H en metros) versus Caudal Volumétrico (Q en $\frac{m^3}{h}$) a una velocidad de giro nominal (N_{nom}).

Si la simulación permite que la bomba varíe su velocidad (por ejemplo, controlada por un variador de frecuencia o por la inercia del motor en un transitorio de parada), usamos las Leyes de Afinidad de las bombas centrífugas para escalar la curva a cualquier velocidad N actual sin necesitar una tabla bidimensional gigante:

$$Q = Q_{\text{tab}} \cdot \left(\frac{N}{N_{\text{nom}}} \right) \Rightarrow W = W_{\text{tab}} \cdot \left(\frac{N}{N_{\text{nom}}} \right) \cdot \frac{\rho}{\rho_{\text{nom}}}$$

$$\Delta P_{\text{bomba}} = \Delta P_{\text{tab}} \cdot \left(\frac{N}{N_{\text{nom}}} \right)^2 \cdot \frac{\rho}{\rho_{\text{nom}}}$$

Esto significa que es posible usar una única lookup table estática 1D ($W_{\text{tab}} \Rightarrow \Delta P_{\text{tab}}$). Para cualquier caudal real W , se «normaliza» a la velocidad nominal, se busca en la tabla la presión nominal, y luego se escala el resultado.

1.2.2. Implementación de la Lookup Table 1D Lineal en Rust

Para mantener las miles de consultas por segundo, la lookup table debe hacer una búsqueda binaria rápida e interpolación lineal, evitando cualquier asignación de memoria dinámica (alloc) en el loop.

```
pub struct LookupTable1D {
    x: Vec<f64>, // Caudales máxicos de prueba (deben estar ordenados)
    y: Vec<f64>, // Saltos de presión correspondientes (Pa)
}

impl LookupTable1D {
    pub fn new(x: Vec<f64>, y: Vec<f64>) -> Self {
        assert_eq!(x.len(), y.len());
        Self { x, y }
    }
}
```

```

/// Interpolación lineal con extrapolación segura en los extremos
pub fn interpolate(&self, target_x: f64) -> f64 {
    let n = self.x.len();
    if target_x <= self.x[0] { return self.y[0]; }
    if target_x >= self.x[n - 1] { return self.y[n - 1]; }

    // Búsqueda binaria optimizada (O(log N))
    let idx = match self.x.binary_search_by(|val|
val.partial_cmp(&target_x).unwrap()) {
        Ok(exact_idx) => exact_idx,
        Err(insert_idx) => insert_idx - 1,
    };

    // Interpolación lineal estándar
    let x0 = self.x[idx];
    let x1 = self.x[idx + 1];
    let y0 = self.y[idx];
    let y1 = self.y[idx + 1];

    y0 + (target_x - x0) * (y1 - y0) / (x1 - x0)
}
}

```

1.2.3. Estructura e Implementación del Bloque Bomba

Para que la bomba sea numéricamente estable, volvemos a aplicar el truco del Semi-Implicito que usamos en el caño. Sin embargo, hay un detalle crítico: la curva de la bomba tiene pendiente negativa (a más caudal, menos presión). Esto actúa como un amortiguador de forma natural, pero si la bomba se apaga o el flujo se invierte, las pérdidas internas (fricción de rotor parado) deben predominar para frenar el fluido.

```

pub struct CentrifugalPumpBlock {
    // Geometría interna equivalente (Inercia del fluido dentro de la carcasa)
    inertia_gain: f64, // (dt * A) / L

    // Curva característica a velocidad nominal
    curve_table: LookupTable1D,
    n_nominal: f64, // RPM nominales (ej. 1500.0)
    rho_nominal: f64, // Densidad de diseño (ej. 1000.0)

    // Pérdida de carga pasiva (cuando la bomba actúa como restricción)
    fric_factor_pasivo: f64,

    // Memoria de estado z^-1
    w_last: f64,
}

impl CentrifugalPumpBlock {
    pub fn tick(&mut self, p_in: f64, p_out: f64, rho: f64, speed: f64, dt: f64) -> f64 {
        let delta_p_headers = p_in - p_out;
    }
}

```

```

let speed_ratio = speed / self.n_nominal;
let rho_ratio = rho / self.rho_nominal;

let dp_bomba = if speed_ratio.abs() > 1e-3 && self.w_last >= 0.0 {
    // --- MODO ACTIVO (Flujo directo y bomba girando) ---
    let w_nominal_look = self.w_last / (speed_ratio * rho_ratio);
    let dp_nominal_look = self.curve_table.interpolate(w_nominal_look);
    dp_nominal_look * speed_ratio.powi(2) * rho_ratio
} else {
    // --- MODO PASIVO O FLUJO INVERSO ---
    0.0
};

// --- ECUACIÓN DE MOMENTO SEMI-IMPLÍCITA ---
let numerator = self.w_last + self.inertia_gain * (delta_p_headers +
dp_bomba);

let w_mod = f64::max(self.w_last.abs(), 1e-4);
let k_fric = if self.w_last >= 0.0 { self.fric_factor_pasivo } else
{ self.fric_factor_pasivo * 5.0 };
let denominator = 1.0 + self.inertia_gain * (k_fric / rho) * w_mod;

let w_current = numerator / denominator;
self.w_last = w_current;
w_current
}
}

```

1.2.4. Acoplamiento de Energía (Transporte Térmico)

La bomba genera un trabajo sobre el fluido que incrementa su energía. Al igual que con el caño, el transporte de entalpía hacia los headers se calcula afuera de la ecuación de momento usando el Upwind Suavizado:

- Flujo Directo ($W > 0$): La bomba succiona entalpía del Header_In y se la inyecta al Header_Out. Físicamente, parte de la potencia consumida por la bomba que no se convierte en presión (ineficiencia térmica o fricción interna) se disipa directamente como calor en el fluido. Se puede sumar esa ineficiencia como un término $Q_{\text{disipado}} = \text{Potencia} \cdot (1 - \eta)$ directo al flujo de energía que recibe el Header_Out.
- Flujo Inverso ($W < 0$): Si las presiones externas vencen a la bomba y el fluido retrocede, la entalpía viaja desde el Header_Out hacia el Header_In.

2. Componentes Volumétricos de Acumulación (Capacitancias)

2.1. Bloque Header (Volumen de Control Capacitivo)

Este documento detalla la arquitectura, el modelado matemático y la implementación en Rust del componente Header (o nodo volumétrico de acumulación). En una simulación termohidráulica por diagramas de bloques distribuidos, el Header actúa como una capacitancia acoplada. Es el encargado de absorber los cambios bruscos de caudal impuestos por las resistencias (Pipe1D, válvulas, bombas), actuando como un amortiguador numérico que «ablanda» las ecuaciones de presión y energía del sistema discreto.

2.1.1. Rol Arquitectónico y Topología

A diferencia del caño discretizado espacialmente, el Header es un modelo de parámetros concentrados (0D).

Variables de Estado: Es el dueño de la presión (p), la entalpía (h) y la densidad (ρ) en un punto de conexión común.

Conectividad: Funciona como un nodo central («hub») al que se pueden conectar M componentes de flujo. No presupone una dirección única; recolecta caudales de masa (W) y flujos de energía (Φ) netos provenientes de todas sus fronteras conectadas, aplicando una convención de signos estricta: flujo entrante es positivo (+), flujo saliente es negativo (-).

2.1.2. Modelado Matemático y Estrategia de Ablandamiento

Para resolver el estado en tiempo discreto sin recurrir a un solver implícito global, el Header expande sus ecuaciones de conservación analíticamente a través de derivadas parciales de la densidad.

2.1.2.1. Conservación de Masa y Energía

Las ecuaciones continuas que gobiernan el volumen fijo V del Header son:

$$\frac{dM}{dt} = \sum W_{\text{in}} - \sum W_{\text{out}} = W_{\text{net}}$$

$$\frac{dU}{dt} = \sum (W \cdot h)_{\text{in}} - \sum (W \cdot h)_{\text{out}} + Q = \Phi_{\text{net}} + Q$$

Para fluidos monofásicos líquidos a presiones moderadas, aproximamos la energía interna total como $U \approx M \cdot h$. Expandiendo la derivada temporal de la energía por la regla del producto:

$$\frac{dU}{dt} = M \frac{dh}{dt} + h \frac{dM}{dt} = (V \cdot \rho) \frac{dh}{dt} + h \cdot W_{\text{net}}$$

Despejando directamente el avance de la entalpía específica:

$$\frac{dh}{dt} = \frac{\Phi_{\text{net}} + Q - h \cdot W_{\text{net}}}{V \cdot \rho}$$

2.1.2.2. El Lazo de Presión Ablandada

Para actualizar la presión de forma explícita y estable, expresamos el cambio de densidad en función de sus coordenadas termodinámicas $\rho(p, h)$:

$$\frac{d\rho}{dt} = \left(\frac{\partial \rho}{\partial p} \right)_h \frac{dp}{dt} + \left(\frac{\partial \rho}{\partial h} \right)_p \frac{dh}{dt}$$

Como $\frac{d\rho}{dt} = \frac{W_{\text{net}}}{V}$, podemos sustituir y despejar la derivada temporal de la presión ($\frac{dp}{dt}$):

$$\frac{dp}{dt} = \frac{1}{V \cdot (\partial \rho / \partial p)_h} \left[W_{\text{net}} - V \cdot \left(\partial \rho / \partial h \right)_p \frac{dh}{dt} \right]$$

1

2.1.3. Estructura de Datos en Rust

El bloque almacena su geometría, sus variables de estado integradas y los coeficientes de acoplamiento termodinámico.

```
pub struct HeaderBlock {
    // Geometría fija del nodo
    volume: f64,

    // VARIABLES DE ESTADO (Salidas del bloque hacia los caños)
    pub p: f64,    // Presión actual (Pa)
    pub h: f64,    // Entalpía específica actual (J/kg)
    pub rho: f64,  // Densidad actual (kg/m³)

    // COEFICIENTES DE ACOPLAMIENTO (Ablandamiento)
    drho_dp_virtual: f64, // (∂ρ/∂p)_h modificado para estabilidad
    drho_dh: f64,         // (∂ρ/∂h)_p real o aproximado
}

=== Implementación del Bloque (Método tick)

impl HeaderBlock {
    pub fn new(volume: f64, initial_p: f64, initial_h: f64) -> Self {
        let mut header = Self {
            volume,
            p: initial_p,
            h: initial_h,
            rho: 1000.0,
            drho_dp_virtual: 4.5e-10 * 1000.0,
            drho_dh: -0.0002 * 1000.0,
        };
        header.refresh_thermo_properties();
        header
    }

    pub fn tick(&mut self, w_net: f64, wh_net: f64, q_externo: f64, dt: f64) {
        let mass = self.volume * self.rho;

        // 1. INTEGRACIÓN DISCRETA DE LA ENERGÍA (Entalpía)
        let net_energy_flow = wh_net + q_externo;
        let dh_dt = (net_energy_flow - (self.h * w_net)) / mass;
        self.h += dh_dt * dt;
    }
}
```

¹Interpretación Física: El término $V \cdot \left(\partial \rho / \partial h \right)_p \frac{dh}{dt}$ captura la expansión térmica confinada. Si el líquido se calienta rápido ($\frac{dh}{dt} > 0$), la densidad tiende a bajar, lo que se traduce en una inyección de presión adicional dentro del volumen cerrado.

Sintonía Numérica: Multiplicar el término de compresibilidad isotérmica $\left(\partial \rho / \partial p \right)_h$ por un factor de ganancia virtual mayor a 1 actúa como un filtro paso bajo para las ondas de presión. Esto «ablanda» el golpe de ariete numérico y permite correr la simulación con un Δt significativamente mayor.

```

// 2. INTEGRACIÓN DISCRETA DE LA PRESIÓN
let thermal_expansion_buffer = self.volume * self.drho_dh * dh_dt;
let dp_dt = (w_net - thermal_expansion_buffer) / (self.volume *
self.drho_dp_virtual);
self.p += dp_dt * dt;

// 3. SINCRONIZACIÓN TERMODINÁMICA
self.refresh_thermo_properties();
}

#[inline(always)]
fn refresh_thermo_properties(&mut self) {
    // Enlace real a librería iapws97 u otra tabla de agua líquida
    self.rho = 1000.0;
    let drho_dp_real = 4.5e-10 * self.rho;
    let coef_ablandamiento = 50.0;
    self.drho_dp_virtual = drho_dp_real * coef_ablandamiento;
    self.drho_dh = -0.0002 * self.rho;
}
}

```

2.1.4. Acoplamiento de Frontera y Flujo Bidireccional

El Header no necesita conocer qué componentes están conectados a él. Su interfaz expone de forma pública sus tres salidas actualizadas: p , h y ρ . La bidireccionalidad se resuelve en las conexiones gracias al esquema de transporte Upwind implementado en los caños:

Si un caño genera un caudal hacia afuera del Header, el caño lee la entalpía h del Header para calcular la energía que se está llevando.

Si un caño empuja fluido hacia adentro del Header, el caño le inyectará su propia entalpía de salida. El Header recibirá un w_{net} positivo y un wh_{net} positivo, absorbiendo la masa y mezclando la energía de forma automática.

2.2. Open Tank (Tanque de Presión Hidrostática)

Un tanque abierto (o pileta de nivel variable) es la simplificación más noble del bloque volumétrico. Al estar en contacto directo con la atmósfera, la cámara de gas desaparece de las ecuaciones porque su presión ya no depende de la compresión del volumen: la presión en la superficie es una constante fija (P_{atm}).

Esto significa que el colchón de aire ya no actúa como un resorte neumático. Toda la variación de presión en el fondo del tanque se debe única y exclusivamente al peso de la columna de líquido (la altura del nivel).

2.2.1. El Modelado Matemático

El modelo se reduce a la conservación de masa y al cálculo de la presión hidrostática pura.

2.2.1.1. Balance de Masa y Nivel

La masa acumulada (M_L) sigue gobernada por la suma algebraica de los caudales de los caños conectados (W_{net}):

$$M_L^{n+1} = M_L^n + W_{\text{net}} \cdot \Delta t$$

A partir de la masa y la densidad actual del líquido (ρ_L), determinamos el volumen y la altura del nivel (z):

$$V_L = \frac{M_L}{\rho_L}$$

$$z = \frac{V_L}{A}$$

2.2.1.2. Ecuación de Presión en la Base

La presión absoluta en el fondo del tanque que «ven» los caños acoplados es simplemente:

$$P_{\text{base}} = P_{\text{atm}} + \rho_L \cdot g \cdot z$$

²

2.2.2. Estructura de Datos en Rust

```
pub struct OpenTankBlock {
    // Geometría fija
    area: f64,    // Área transversal (m²)
    g: f64,       // Gravedad (m/s²)
    p_atm: f64,   // Presión atmosférica local (Pa)

    // VARIABLES DE ESTADO INTEGRADAS
    pub m_liq: f64, // Masa de líquido actual (kg)
    pub h_liq: f64, // Entalpía específica del líquido (J/kg)

    // SALIDAS DEDUCIDAS
    pub p_base: f64, // Presión en el fondo (Pa)
```

²Nota Numérica: En simulaciones numéricas, un tanque abierto es el estabilizador definitivo. Al fijar $P_{\text{surface}} = P_{\text{atm}}$, el sistema tiene un nodo de presión de referencia constante. Esto evita que las presiones del sistema floten o diverjan sin control numérico.

```

    pub level: f64, // Nivel de líquido (m)
    pub rho_l: f64, // Densidad del líquido (kg/m³)
}

=== Implementación del Bloque (tick)

impl OpenTankBlock {
    pub fn new(area: f64, initial_level: f64, p_atm: f64, initial_h: f64) -> Self {
        let rho_l = 1000.0;
        let v_liq = initial_level * area;
        let m_liq = v_liq * rho_l;

        let mut tank = Self {
            area,
            g: 9.80665,
            p_atm,
            m_liq,
            h_liq: initial_h,
            p_base: p_atm + rho_l * 9.80665 * initial_level,
            level: initial_level,
            rho_l,
        };
        tank.refresh_geometry();
        tank
    }

    pub fn tick(&mut self, w_net: f64, wh_net: f64, q_externo: f64, dt: f64) {
        // 1. INTEGRACIÓN DE MASA
        self.m_liq += w_net * dt;
        if self.m_liq < 0.0 { self.m_liq = 0.0; }

        // 2. INTEGRACIÓN DE ENERGÍA
        let dh_dt = (wh_net + q_externo - (self.h_liq * w_net)) / (self.m_liq +
1e-3);
        self.h_liq += dh_dt * dt;

        // 3. ACTUALIZACIÓN DE SALIDAS
        self.refresh_geometry();
    }

    fn refresh_geometry(&mut self) {
        self.rho_l = 1000.0; // Enlace simplificado
        let v_liq = self.m_liq / self.rho_l;
        self.level = v_liq / self.area;
        self.p_base = self.p_atm + self.rho_l * self.g * self.level;
    }
}

```

2.2.3. Particularidades en la Red

Desborde Teórico: Si la masa inyectada supera la capacidad física, el modelo aumentará el level indefinidamente. Si se desea modelar el derrame, se debe limitar el level a la altura constructiva máxima en refresh_geometry() y reportar la masa perdida fuera del balance.

Evaporación y Pérdidas Térmicas (Q_{externo}): Los tanques abiertos tienen gran superficie de intercambio. El término Q_{externo} permite simular la pérdida de calor por convección con el aire o radiación solar.

2.3. Tank Closed (Tanque Presurizado de Nivel Variable)

Un tanque de nivel variable cerrado se diferencia del Header estructuralmente en un aspecto crítico: es un sistema de volumen de fluido variable con interfaz líquido-gas.

Mientras que el Header es un nodo totalmente lleno (monofásico cerrado, donde la presión sube instantáneamente al inyectar masa), el tanque cerrado absorbe masa cambiando su nivel de líquido, comprimiendo o expandiendo el colchón de gas (aire o nitrógeno) atrapado en la parte superior. Por lo tanto, la presión se rige por la ley de los gases ideales y procesos politrópicos.

2.3.1. El Modelado Matemático (0D de Dos Zonas)

Dividimos el tanque de volumen total fijo (V_{tot}) en dos zonas de volumen variable:

Zona Líquida (V_L): Fluido incompresible con densidad ρ_L .

Zona de Gas (V_G): Gas ideal compresible con masa fija M_G .

2.3.1.1. Balance de Masa y Geometría

La masa de líquido varía según el caudal neto de los caños conectados (W_{net}):

$$\frac{dM_L}{dt} = W_{\text{net}}$$

Conocida la masa de líquido, calculamos los volúmenes y el nivel hidrostático (z) si el tanque tiene un área transversal constante (A):

$$V_L = \frac{M_L}{\rho_L}$$

$$z = \frac{V_L}{A}$$

$$V_G = V_{\text{tot}} - V_L$$

2.3.1.2. Dinámica de Presión (Compresión del Gas)

Asumiendo que el gas se comprime de forma adiabática (proceso rápido sin transferencia de calor, $\gamma = 1.4$ para el aire) o isotérmica ($\gamma = 1.0$), la presión del gas P_G evoluciona como:

$$P_G \cdot V_G^\gamma = C \Rightarrow P_G = P_{G,0} \cdot \left(\frac{V_{G,0}}{V_G} \right)^\gamma$$

2.3.1.3. Presión en el Fondo del Tanque

La presión que «ven» los caños conectados en la base incluye la presión del colchón de gas más el peso de la columna líquida:

$$P_{\text{base}} = P_G + \rho_L \cdot g \cdot z$$

2.3.2. Estructura de Datos en Rust

```
pub struct TankBlock {
    v_total: f64,
    area: f64,
    g: f64,

    // Colchón de gas constante
```

```

    m_gas: f64,
    r_gas: f64,
    t_gas: f64,
    gamma: f64,
    p_gas_nominal: f64,
    v_gas_nominal: f64,

    // VARIABLES DE ESTADO INTEGRADAS
    pub m_liq: f64,
    pub h_liq: f64,

    // SALIDAS DEDUCIDAS
    pub p_base: f64,
    pub level: f64,
    pub rho_l: f64,
}

=== Implementación del Bloque (tick)

impl TankBlock {
    pub fn new(v_total: f64, area: f64, initial_level: f64, initial_p_gas: f64,
initial_h: f64) -> Self {
        let rho_l = 1000.0;
        let v_liq = initial_level * area;
        let v_gas = v_total - v_liq;
        let m_liq = v_liq * rho_l;

        Self {
            v_total,
            area,
            g: 9.81,
            m_gas: 0.0,
            r_gas: 287.05,
            t_gas: 293.15,
            gamma: 1.4,
            p_gas_nominal: initial_p_gas,
            v_gas_nominal: v_gas,
            m_liq,
            h_liq: initial_h,
            p_base: initial_p_gas + rho_l * 9.81 * initial_level,
            level: initial_level,
            rho_l,
        }
    }

    pub fn tick(&mut self, w_net: f64, wh_net: f64, q_externo: f64, dt: f64) {
        // 1. INTEGRACIÓN DE MASA
        self.m_liq += w_net * dt;
        if self.m_liq < 0.0 { self.m_liq = 0.0; }
        let max_mass = self.v_total * self.rho_l * 0.99;
        if self.m_liq > max_mass { self.m_liq = max_mass; }

        // 2. INTEGRACIÓN DE ENERGÍA

```

```

        let dh_dt = (wh_net + q_externo - (self.h_liq * w_net)) / (self.m_liq +
le-3);
        self.h_liq += dh_dt * dt;

        // 3. ACTUALIZACIÓN DE SALIDAS
        self.refresh_geometry();
    }

    fn refresh_geometry(&mut self) {
        self.rho_l = 1000.0;
        let v_liq = self.m_liq / self.rho_l;
        let v_gas = self.v_total - v_liq;
        self.level = v_liq / self.area;

        // Ley politrópica del gas
        let p_gas = self.p_gas_nominal * (self.v_gas_nominal /
v_gas).powf(self.gamma);
        self.p_base = p_gas + self.rho_l * self.g * self.level;
    }
}

```

2.4. Tanque Térmico Estratificado

Un tanque estratificado (típico en sistemas termosolares, acumulación térmica o agua caliente sanitaria) se basa en el principio de que el agua caliente es menos densa que el agua fría. Esto genera capas térmicas estables que no se mezclan de forma natural debido a la flotabilidad.

En un esquema de diagramas de bloques distribuidos, se modela como una serie de sub-volúmenes (nodos) acoplados verticalmente. Cada capa actúa matemáticamente como un Header o un pequeño tanque interconectado, pero aplicando dos leyes físicas adicionales: convección forzada (el agua se desplaza entre capas al entrar/salir fluido) y mezclado/conducción (intercambio destructivo de la estratificación).

2.4.1. Topología de N Capas ($N = 2, 3, 4...$)

Dividir el tanque en 3 o 4 capas suele ser el punto óptimo para simulación en tiempo real.

2 capas (Termoclina simple): Divide el tanque en «Capa Superior Hot» y «Capa Inferior Cold». Es rápido pero no captura bien los frentes térmicos móviles.

3 a 4 capas: Permite modelar una zona de transición (termoclina) que sube y baja según la carga y descarga del tanque.

```

+-----+
|  Capa 1 (Top / Hot)  | <- Entrada/Salida Alta (Caldera/Consumo)
+-----+
|      Capa 2          | | Flujos de acoplamiento interno (W_ij)
+-----+               | Conducción térmica (Q_cond)
|      Capa 3          | v Mezclado flotante (Q_turb)
+-----+
|  Capa 4 (Bottom/Cold) | <- Entrada/Salida Baja (Retorno)
+-----+

```

Cada capa i tiene su propia masa M_i , entalpía h_i y temperatura T_i . El volumen geométrico de cada capa es fijo ($V_i = \frac{V_{\text{tot}}}{N}$).

2.4.2. Modelado de los Fenómenos de Mezclado

Hay tres mecanismos que transfieren energía entre la capa i y la capa $i + 1$:

2.4.2.1. Convección Forzada (Desplazamiento Neto de Masa)

Si se inyecta agua en la Capa 4 (abajo) y se extrae agua de la Capa 1 (arriba), se genera un caudal neto vertical ascendente (W_{net}) que atraviesa todas las capas intermedias. Cada capa le transfiere masa a la siguiente usando el esquema Upwind: si el flujo sube, la capa i recibe la entalpía de la capa $i + 1$.

2.4.2.2. Conducción Térmica Pasiva

Existe un flujo conductivo directo entre capas adyacentes debido a la diferencia de temperatura, que tiende a homogeneizar el tanque lentamente:

$$Q_{\text{cond}, i \rightarrow i+1} = \frac{k \cdot A}{\Delta x} (T_i - T_{i+1})$$

Donde k es la conductividad térmica del agua, A es el área transversal del tanque y Δx es la distancia entre los centros de las capas.

2.4.2.3. Mezclado por Inestabilidad del Flujo (Inversión de Densidad)

Si ingresa agua fría en la capa superior o agua caliente en la inferior, el sistema se vuelve hidrodinámicamente inestable. El agua pesada cae y la liviana sube, generando mezclado turbulento macroscópico.

Modelado: Si $T_{i+1} > T_i$ (la capa de abajo está más caliente que la de arriba), calculamos un coeficiente de mezclado masivo (k_{mix}) que transfiere energía de forma casi instantánea hasta que las densidades se ordenen.

2.4.3. Implementación en Rust (Modelo de 3 Capas)

```
pub struct StratifiedTank {
    volume_layer: f64, // Volumen de cada capa (m³)
    area: f64,         // Área transversal (m²)
    dx: f64,           // Distancia entre centros de capas (m)

    // Variables de estado [0: Top, 1: Mid, 2: Bottom]
    pub p: f64,
    pub h: [f64; 3],
    pub t: [f64; 3],
    pub rho: [f64; 3],

    // Parámetros de mezcla
    k_water: f64,      // Conductividad térmica del agua (~0.6 W/mK)
    mix_factor: f64,   // Multiplicador de mezcla turbulenta por inversión
}

impl StratifiedTank {
    pub fn new(total_volume: f64, area: f64, initial_p: f64, initial_t: f64) -> Self {
        let vol_layer = total_volume / 3.0;
        let dx = vol_layer / area;

        Self {
            volume_layer: vol_layer,
            area,
            dx,
            p: initial_p,
            h: [4184.0 * (initial_t - 273.15); 3],
            t: [initial_t; 3],
            rho: [1000.0; 3],
            k_water: 0.6,
            mix_factor: 50.0,
        }
    }

    pub fn tick(&mut self, w_top: f64, wh_top: f64, w_bot: f64, wh_bot: f64, dt: f64) {
        let w_internal = w_top;
```

```

let q_inter_01 = self.calc_layer_exchange(0, 1);
let q_inter_12 = self.calc_layer_exchange(1, 2);

// --- CAPA 0 (TOP) ---
let mass_0 = self.volume_layer * self.rho[0];
let conv_01 = if w_internal > 0.0 { -w_internal * self.h[0] } else { -
w_internal * self.h[1] };
let dh0_dt = (wh_top + conv_01 - q_inter_01) / mass_0;

// --- CAPA 1 (MID) ---
let mass_1 = self.volume_layer * self.rho[1];
let conv_10 = if w_internal > 0.0 { w_internal * self.h[0] } else
{ w_internal * self.h[1] };
let conv_12 = if w_internal > 0.0 { -w_internal * self.h[1] } else { -
w_internal * self.h[2] };
let dh1_dt = (conv_10 + conv_12 + q_inter_01 - q_inter_12) / mass_1;

// --- CAPA 2 (BOTTOM) ---
let mass_2 = self.volume_layer * self.rho[2];
let conv_21 = if w_internal > 0.0 { w_internal * self.h[1] } else
{ w_internal * self.h[2] };
let dh2_dt = (wh_bot + conv_21 + q_inter_12) / mass_2;

self.h[0] += dh0_dt * dt;
self.h[1] += dh1_dt * dt;
self.h[2] += dh2_dt * dt;

self.refresh_properties();
}

fn calc_layer_exchange(&self, i: usize, j: usize) -> f64 {
    let q_cond = (self.k_water * self.area / self.dx) * (self.t[i] -
self.t[j]);

    let q_mix = if self.t[j] > self.t[i] {
        (self.t[j] - self.t[i]) * (self.k_water * self.area / self.dx) *
self.mix_factor
    } else {
        0.0
    };

    q_cond - q_mix
}

fn refresh_properties(&mut self) {
    for i in 0..3 {
        self.t[i] = 273.15 + (self.h[i] / 4184.0);
        self.rho[i] = 1000.0 * (1.0 - 0.0002 * (self.t[i] - 293.15));
    }
}
}

```